

المحاضرة الرابعة: طرق التعامل مع classes وطرق الوراثة في اطار Dart



م. سليمان الشوصي

محتويات المحاضرة Contents

- ❖ الأصناف والدوال المجردة (Abstract Class & Methods)
- ❖ الواجهات والوراثة المتعددة (Interfaces & implements)
- ❖ إعادة استخدام السلوكيات (Mixins & with)
- ❖ التعداد Enumeration (enum)
- ❖ سلامة القيم الفارغة (Null Safety)
- ❖ الأنماط المعممة (Generics)
- ❖ الاستثناءات (Exceptions) والأخطاء (Errors)
- ❖ رمي الاستثناء Throwing Exceptions

```

abstract class A {
  showInfo(); // دالة abstract
}
class B extends A {
  @override
  showInfo() {
    print("Abstract methode");
  }
}
Run | Debug
void main() {
  var a = A();
  var b = B();
  b.showInfo();
}

```

❖ **الصف المجرّد (Abstract Class):** في البرمجة كائنية التوجّه (OOP)، تُعدّ Abstract Class بمثابة "عقد برمجيات" (Contract) أو "قالب عام غير مكتمل".

يُستخدم الصف المجرّد عندما نريد وضع هيكل أساسي وقواعد عامة تلتزم بها مجموعة من الكلاسات الأبناء، دون أن نحدد التفاصيل الدقيقة لكيفية التنفيذ؛ حيث نترك لكل كلاس ابن حرية تنفيذ التفاصيل بطريقته الخاصة.

❖ **القواعد الأساسية للـ Abstract Class ثلاث وهي:**

١. **منع الإنشاء المباشر (No Direct Instantiation)**

- **القاعدة:** لا يمكن إنشاء كائن (Object) مباشر من الكلاس الـ abstract باستخدام new أو الاستدعاء المباشر مثل `var a = A();`.
- **السبب:** لأنه صنف تجريدي ووظيفته أن يكون مجرد مخطط أو قالب عام لم يتم اكتمال تفاصيله بعد.

٢. **إلزامية التنفيذ وإعادة التعريف (Mandatory Implementation)**

- **القاعدة:** عند إنشاء كلاس ابن عادي يرث (extends) من كلاس abstract؛ **يجب عليه حتماً** إعادة تعريف وتنفيذ جميع الدوال المجرّدة (@override) الموجودة في الأب.
- **في الكود:** الكلاس B كان مجبراً على كتابة الجسم والتنفيذ الفعلي لدالة `showInfo()`.

٣. **شرط وتبعية الدالة المجرّدة (Abstract Method Requirement)**

- **القاعدة:** الدالة المجرّدة (Abstract Method) هي دالة تُكتب بدون جسم/تنفيذ (بدون { } وتنتهي بـ ;). ولا يجوز كتابة دالة مجردة إلا داخل كلاس معرف بـ `abstract`.
- **بمعنى آخر:** إذا احتوى الكلاس على دالة مجردة واحدة على الأقل، يجب معرفة الكلاس ككل بـ `abstract class`.

```
// 1. الكلاس المجرد (العقد الأساسي)
abstract class Shape {
  // بدون body) دالة مجردة لحساب المساحة
  double calculateArea();
}

// 2. كلاس الابن الأول: الدائرة
class Circle extends Shape {
  double radius;
  Circle(this.radius);
  @override
  double calculateArea() {
    return 3.14 * radius * radius;
  }
}

// 3. كلاس الابن الثاني: المربع
class Square extends Shape {
  double side;
  Square(this.side);
  @override
  double calculateArea() {
    return side * side;
  }
}

Run | Debug
void main() {
  var c = Circle(5);
  var s = Square(4);
  print("Circle area= ${c.calculateArea()}");
  print("Square area= ${s.calculateArea()}");
}
```

❖ الهدف والفائدة الأساسية

- التوحيد والتنظيم (Standardization): إجبار جميع المطورين أو الكلاسات المشتقة على اتباع أسماء ودوال موحدة داخل المشروع.
- إخفاء التباين والتفاصيل (Abstraction): التركيز على "ماذا تفعل الدالة" بدلاً من "كيف تفعل ذلك".
- الحد من الأخطاء: منع إنشاء كائنات مبهمه ليس لها معنى مستقل في الذاكرة (فلا يوجد كيان مجرد في الواقع اسمه "شكل" فقط دون أن يكون دائرة أو مربعاً).

❖ على سبيل المثال: لو اردنا تصمم كلاس عام اسمه Shape (شكل هندسي):

- جميع الأشكال الهندسية لها دالة حساب المساحة calculateArea().
- لكن هل يمكنك كتابة قانون رياضي موحد للمساحة ينطبق على الدائرة والاطار والمربع في نفس الوقت داخل الكلاس Shape؟ بالطبع لا!
- لذلك، ننشئ Shape كـ abstract class ونضع فيه دالة مجردة calculateArea() بدون جسم أو قانون.
- ثم يأتي كلاس الابن Circle وينفذ القانون الخاص بالدائرة، بينما يأتي كلاس الابن Square وينفذ القانون الخاص بالمربع.

```
//الوراثة المتعددة من كلاس
//implements استخدام
abstract class A {
  showInfo();
}
abstract class B {
  showInfo2() {
    print("Abstract methode");
  }
}
class C implements A, B {
  @override
  showInfo() {
    print("interface class A,B");
  }
  @override
  showInfo2() {
    print("Suliman");
  }
}
Run | Debug
void main() {
  var c = C();
  c.showInfo();
  c.showInfo2();
}
interface class A,B
Suliman
```

❖ **الوراثة المتعددة (Multiple Inheritance):** تُطبق الكلاسات في الكود الوراثة المتعددة عن طريق الواجهات (Multiple Interface Inheritance)؛ حيث إن لغة Dart لا تدعم الوراثة المتعددة المباشرة باستخدام الكلمة المفتاحية extends (أي يمنع كتابة class C extends A, B)، ولكنها تُتيح لكلاس واحد أن يطبق أكثر من واجهة بنفس الوقت باستخدام الكلمة المفتاحية implements.

- ❖ **الواجهة (Interface):** هو abstract class لا يمكن انشاء الـ object منه.
- المفهوم العام: الواجهة هي عقد برمجي (Contract) يُحدد "ماذا يجب على الكلاس أن يفعل" (قائمة بالدوال والخصائص المطلوب توفرها)، دون الاهتمام بـ "كيفية التنفيذ".
 - في إطار Dart: لا توجد كلمة مفتاحية خاصة باسم interface؛ بل كل كلاس في Dart يعتبر بشكل ضمني (Implicit Interface) واجهة يمكن لكلاسات أخرى تطبيقها.

تطبيق الواجهة (implements):

- المفهوم العام: الكلمة المفتاحية implements تُستخدم عندما يُقرر كلاس التزام وتطبيق عقد واجهة معينة (أو عدة واجهات يفصل بينها بفواصل مثل implements A, B).
- القاعدة الذهبية لـ implements: الكلاس الابن الذي يستخدم implements مُجبر على إعادة تعريف وتنفيذ جميع الدوال والخصائص المعرّفة في الواجهات، حتى وإن كانت تلك الدوال تحتوي على جسم وتنفيذ أكواد جاهزة في الكلاس الأب (كما حدث مع الدالة showInfo2 في الكلاس B).

وجه المقارنة	extends الوراثة الاعتيادية	implements تطبيق الواجهات
عدد الكلاسات	يرث من كلاس واحد فقط (Single Inheritance).	يطبق عدة واجهات في نفس الوقت. (A, B, C...)
إعادة التعريف	يعيد تعريف الدوال المجردة فقط، ويرث الكود الجاهز كما هو.	مُجبر على إعادة تعريف كل الدوال حتى اللاتي تحتوي على كود جاهز في الأب.
الهدف	مشاركة وإعادة استخدام الكود (Code Reuse).	توحيد الهيكل والالتزام بالعقود. (Contract Enforcement)

```

mixin A {
  showInfo() {
    print("suliman_1");
  }
}
mixin B {
  showInfo2() {
    print("abdulsalam_1");
  }
}
class C {
  showInfo3() {
    print("Alshossi_1");
  }
}
class D extends C with A, B {
  @override
  showInfo() {
    print("suliman_2");
  }
  @override
  showInfo2() {
    print("abdulsalam_2");
  }
  @override
  showInfo3() {
    print("Alshossi_2");
  }
}
Run | Debug
void main() {
  var d = D();
  d.showInfo();
  d.showInfo2();
  d.showInfo3();
}
suliman_2
abdulsalam_2
Alshossi_2

```

```

mixin A {
  showInfo();
}
mixin B {
  showInfo2();
}
abstract class C {
  showInfo3();
}
class D extends C with A, B {
  @override
  showInfo() {
    print("suliman_2");
  }
  @override
  showInfo2() {
    print("abdulsalam_2");
  }
  @override
  showInfo3() {
    print("Alshossi_2");
  }
}
Run | Debug
void main() {
  var d = D();
  d.showInfo();
  d.showInfo2();
  d.showInfo3();
}

```

❖ **مفهوم الـ (المزج او الخلط) Mixin في Dart:** هو كلاس يحتوي على دوال وخصائص جاهزة ومكتملة التنفيذ، يُستخدم لإضافة وتضمين هذه السلوكيات إلى كلاسات أخرى دون الحاجة لوراثة مباشرة (extends) وبدون إجبار الكلاس على إعادة كتابة الكود (implements).

- الكلمة المفتاحية للتعريف: يُعرّف باستخدام mixin (أو mixin class).
- الكلمة المفتاحية للدمج: يُدمج مع الكلاسات الأخرى باستخدام الكلمة المفتاحية with

الغرض والفائدة الأساسية من استخدامه:

- إعادة استخدام الكود بين كلاسات غير مرتبطة بوراثة (Code Reuse): تتيح لك مشاركة وظيفة معينة (مثل: إرسال التنبيهات، تسجيل البيانات Logging، أو التحقق من الاتصال بالإنترنت) بين عدة كلاسات تقع في أفرع مختلفة تماماً من شجرة الوراثة.
- تجنب مشاكل الوراثة المتعددة (Multiple Inheritance Problem): تمنحك مميزات الوراثة المتعددة (استخدام كود جاهز من عدة مصادر) دون الوقوع في مشاكل التعارض التعقيد للوراثة المتعددة التقليدية.

مقارنة سريعة

المفهوم	الكلمة المفتاحية	هل يرث الكود الجاهز؟	هل يدعم التعدد؟
Inheritance	extends	نعم	كلاس واحد فقط
Interface	implements	لا (مُجبر على إعادة الكتابة)	نعم (عدة واجهات)
Mixin	with	نعم (يأخذ الكود جاهزاً)	نعم (عدة mixins)

```
enum MyColors { blak, red, yellow, green }
| Ctrl+L to chat, Ctrl+K to generate
Run | Debug
void main() {
  print(MyColors);
  print(MyColors.values);
  print(MyColors.values[0]);
  print(MyColors.values.length);
  List lc = MyColors.values;
  print(lc[2]);
}
```

```
myColors
[myColors.blak, myColors.red, myColors.yellow, myColors.green]
myColors.blak
4
myColors.yellow
```

❖ **مفهوم الـ Enum Class** (اختصار لـ Enumeration التعداد) يُعد من المفاهيم الأساسية والمهمة جداً لجعل الكود أكثر أماناً ونظافة عند التعامل مع القيم الثابتة والمحددة.

الـ Enum: هو نوع بيانات خاص تُعرّف فيه قائمة محددة وثابتة من الخيارات (Enumerate Values) التي لا تتغير.

بدلاً من كتابة الخيارات كنصوص عادية (Strings) أو أرقام (Integers) قد تتغير أو يُخطئ المطور في كتابتها، يمنحك الـ Enum مجموعة مغلقة من الخيارات المحددة مسبقاً.

الغرض والفائدة الأساسية من استخدامه:

- **سلامة وأمان الكود (Type Safety):** حصر الخيارات المتاحة؛ فلا يمكن للمطور أو المستخدم اختيار أو تمرير قيمة غير موجودة ضمن القائمة.
- **منع الأخطاء المطبعية (Typo Prevention):** يتولى المحرر التنبؤ بالخيارات المتاحة تلقائياً، مما يمنع الأخطاء الناجمة عن كتابة الكلمات الإملائية بشكل خاطئ.
- **وضوح الكود وقابليته للقراءة (Readability):** بدلاً من استخدام أرقام مجهولة (مثل: ١، ٢، ٣) لمعرفة حالة معينة، تستخدم أسماء صريحة واضحة المعنى.

أمثلة من واقع التطبيقات (Real-World Examples): تُستخدم الـ Enums بكثرة في بناء واجهات وتطبيقات الهاتف الذكي، ومن أشهر استخداماتها:

- حالة الطلب في تطبيقات التوصيل: Options: [pending, preparing, onTheWay, delivered, cancelled]
- حالات الشاشة والتحميل (State Management): Options: [initial, loading, success, error]
- حالة اتصال المستخدم (User Status): Options: [online, offline, busy, away]
- طرق الدفع الإلكتروني: Options: [creditCard, cashOnDelivery, paypal, applePay]
- مستوى الصلاحيات (User Roles): Options: [admin, user, guest]


```
enum MyColors { blak, red,
                yellow, green }

Run | Debug
void main() {
  print(changeColor(MyColors.red));
  print(changeColor(MyColors.blak));
  print(changeColor(MyColors.green));
}

changeColor(MyColors co) {
  switch (co) {
    case MyColors.blak:
      return "blak";
    case MyColors.red:
      return "red";
    case MyColors.yellow:
      return "yellow";
    case MyColors.green:
      return "green";
  }}

```

```
red
blak
green
```

❖ تُعتبر جملة الاختيار **switch** هي الطريقة المثالية والشائعة جداً للتعامل مع الـ **enum**؛ لأنها تغطي جميع الحالات المتاحة بشكل دقيق وآمن.

النية عمل الكود الموضح في الصورة

١. تحديد نوع المعامل (**MyColors co**):

الدالة **changeColor** تفرض استقبال أي نص عشوائي، وتقبل فقط خياراً معتمداً من نوع **MyColors** (مثل **MyColors.red**).

١. الفحص الشامل عبر **switch**: تقوم **switch** بمطابقة القيمة الممررة (**co**) مع الحالات المعرفة داخل الـ **Enum**. عند إرسال **MyColors.red** في **main()**، تتطابق القيمة مع **case MyColors.red** وترجع النص "red".

الميزة والأمان البرمجي هنا (**Exhaustiveness Check**)

تُستخدم الـ **Enums** بكثرة في بناء واجهات وتطبيقات الهاتف الذكي، ومن أشهر استخداماتها:

- حالة الطلب في تطبيقات التوصيل
- التغطية الكاملة: لغة **Dart** تتأكد من أن **switch** قامت بفحص جميع عناصر الـ **Enum** الأربعة (**blak, red, yellow, green**).
- تنبيه المطور: لو نسيت كتابة **case** لأحد الألوان، سيقوم المترجم (**Compiler**) بوضع خط أحمر وتنبيهك فوراً بأن هناك حالة مفقودة، مما يمنع وقوع الأخطاء أثناء تشغيل التطبيق.

ملاحظة:

❖ استخدام **Enum** مع **switch** يُعطي تحكماً كاملاً بأمان ١٠٠%؛ فلا يمكن تمرير قيمة خاطئة للدالة، ولن تنسى التعامل مع أي حالة من حالات الـ **Enum**.


```

class UserProfile {
  //1. استخدام ? : الاسم اختياري وقد يكون null
  String? nickname;
  // 2. استخدام late :
  // قيمة العمر إجباري لكن سأجلبه لاحقاً
  late int age;
  void setupAge() {
    age = 25;
  }
}

Run | Debug
void main() {
  var user = UserProfile();
  //late تجربة
  user.setupAge();
  print("العمر: ${user.age}");
  // --- ! تجربة ? و ---
  user.nickname = "سليمان";
  int nameLength = user.nickname!.length;
  print("الاسم: ${user.nickname}");
  print("عدد حروف الاسم: $nameLength");
}

```

❖ **Null Safety (سلامة القيم الفارغة):** هي ميزة في لغة Dart تجعل جميع المتغيرات غير قابلة لاستقبال قيمة فارغة (Non-Nullable) بشكل افتراضي، إلا إذا قمت بإعلام المترجم صراحةً بأن هذا المتغير قد يحمل قيمة null.

❖ **معامل التَقْبَل ؟ (Nullable Operator)** إضافة علامة الاستفهام ؟ بعد نوع المتغير تخبر المترجم: "هذا المتغير مسموح له أن يكون فارغاً (null) في أي وقت".

❖ **معامل التوكيد ! (Null Assertion Operator).**

❖ وظيفتها ببساطة هي أن تقول للمترجم (Compiler): "أنا كبرمجي أضمن لك أن هذا المتغير القابل للفراغ (?) يحمل قيمة حالياً وليس null، فتعامل معه كمتغير عادي بدون خوف".

❖ **الكلمة المفتاحية late (Late Initialization)** تخبر المترجم: ان هذا المتغير لا يقبل القيمة الفارغة (Non-Nullable)، ولكنني أعدك بأنني سأعطيه قيمة لاحقاً قبل أن أحاول قراءته أو طباعته.

الغرض والفائدة من هذا الأسلوب:

- حماية التطبيق من الانهيار (Crash Prevention): اكتشاف الأخطاء المتعلقة بالقيم الفارغة أثناء كتابة الكود (Compile-Time) بدلاً من تفاجؤ المستخدم بها أثناء تشغيل التطبيق (Run-Time).
- تحسين أداء الكود: يتيح للمترجم تحسين استهلاك الذاكرة وتسريع التشغيل لأنه يعلم يقيناً أي المتغيرات يحتاج فحص الـ null وأياًها آمن تماماً.

❖ **Generics (الأنماط المعممة):** هي ميزة برمجية تُمكن من كتابة كلاس أو دالة مرنة تتسق مع أنواع مختلفة من البيانات (Data Types) دون تحديد نوع بيانات ثابت أو محدد أثناء مرحلة كتابة الكود الأصلي.

بدلاً من كتابة كلاس مخصص لل String وكلاس آخر مخصص لل int أو double، نستخدم رموزاً بديلة (Type Parameters) مثل T أو E لتكون بمثابة "خانة حجز موقع" (Placeholders) للأنواع التي ستُحدد لاحقاً عند إنشاء الكائن

الغرض من استخدامه:

تُستخدم ال Enums بكثرة في بناء واجهات وتطبيقات الهاتف الذكي، ومن أشهر استخداماتها:

- إعادة استخدام الكود (Code Reusability): كتابة قالب برمجي واحد (Generic Class) يستوعب مختلف أنواع البيانات بدلاً من تكرار الكود.
- سلامة وأمان الأنواع (Type Safety): يكتشف المترجم (Compiler) أي خطأ في نوع البيانات الممررة وقت الكتابة بدلاً من حدوث أخطاء أثناء التشغيل (Runtime Errors).
- تقليل الاعتماد على dynamic: تتيح لنا التعامل مع أنواع مختلفة دون التوضيح بأمان وفحص الأنواع التي توفرها اللغة

```
class A {
  late String x;
}

Run | Debug
void main() {
  var a = A();
  a.x = 2;
  a.x = "Generic";
}
```

```
class A<T> {
  T? x;
}

Run | Debug
void main() {
  var a = A<String>();
  a.x = "Generic";
  print(a.x);

  var p = A();
  p.x = 2026;
  print(p.x);
}
```

```
class A<T, E> {  
  T? x;  
  late E y;  
}  
Run | Debug  
void main() {  
  var a = A<String, double>();  
  a.x = "Generic";  
  a.y = 250.5;  
  print(a.x);  
  print(a.y);  
  
  var p = A<int, bool>();  
  p.x = 2026;  
  p.y = 2026;  
  print(p.x);  
  print(p.y);  
  
  var g = A<int>();  
  g.x = 2026;  
  g.y = false;  
  print(p.x);  
  print(p.y);  
}
```

آلية عمل الكود الموضح في الصورة

١. التعميم في الكلاس :class A<T, E>

تم تعريف الكلاس لتقبل نوعين مختلفين من البيانات غير المحددة مسبقاً (T و E).

المتغير x أصبح من النوع T والمتغير y من النوع E.

٢. التحديد وقت الإنشاء في :main()

- في الكائن الأول: A<String, double> أصبحت T = String و E = double.
- في الكائن الثاني: A <String, bool> أصبحت T = String و E = bool.

```
import 'dart:io';
Run | Debug
void main() {
  print("enter your pirth ");
  String? birth = stdin.readLineSync();
  var age = DateTime.now().year - int.parse(birth!);
  print("Age = $age");
}
```

```
Run | Debug
void main() {
  try {
    print("enter your pirth ");
    String? birth = stdin.readLineSync();
    var age = DateTime.now().year - int.parse(birth!);
    print("Age = $age");
  } catch (e) {
    print("The value Invalid");
    //2 لمعرفة نوع الاخطاء
    // print(e);
  }
  //1-لتحقق من استمرار تنفيذ الكود
  print("Done");
}
```

❖ **ال-Exception:** هو حدث غير متوقع أو خطأ يقع أثناء تشغيل البرنامج (Run-Time) يؤدي إلى توقف التطبيق فجأة وانهيائه (Crash) إذا لم يتم التعامل معه مسبقاً.

Exception هو كلاس يُعبر عن الأخطاء التابعة لمنظومة **Exception** (وليس **Error** الكارثية للنظام)، والتي يُتوقع حدوثها أثناء التشغيل ويمكن التقاطها ومعالجتها بـ **try-catch**.

❖ **معالجة الاستثناءات (Exception Handling)** هي الآلية التي تتيح للبرنامج التقاط هذه الأخطاء والتعامل معها بمرونة، مما يضمن استمرار تشغيل باقي أجزاء التطبيق دون انهيار.

❖ **Try{}:** يتم وضع فيها الكود الخَطِر الذي نخشى وقوع خطأ فيه أثناء التشغيل.

❖ **Catch:** خط الدفاع الذي يستقبل الخطأ وينفذ التطبيق من الانهيار.

```
enter your pirth
kkkkkkkkkkkkkk
The value Invalid
Done
```

```
enter your pirth
kkkkkkkkkkkkkkkkkk
The value Invalid
FormatException: Invalid radix-10 number (at character 1)
kkkkkkkkkkkkkkkkkk
^
Done
```

❖ أنواع الاستثناءات (Exceptions) والأخطاء (Errors) الشائعة التي قد تظهر أثناء تشغيل برنامج Dart.

١. الأخطاء المتعلقة بمعالجة وتحويل البيانات.

- **FormatException**: الذي يحدث عند فشل تحويل صيغ البيانات

٢. أخطاء متعلقة بالعمليات الحسابية والأرقام

- **UnsupportedError / IntegerDivisionByZeroException**:

تحدث عند محاولة تنفيذ عملية غير مدعومة رياضياً، مثل القسمة على صفر باستخدام القسمة الصحيحة (١٠ / ~ ٠).

٣. أخطاء القوائم والمصفوفات (Collections)

- **(RangeError أو IndexOutOfRangeException)**: تحدث بسبب محاولة الوصول إلى عنصر في القائمة عبر دليل (Index) غير موجود (مثلاً: قائمة فيها ٣ عناصر وتحاول الوصول للعنصر رقم ٥).

٤. أخطاء النماذج وتطابق الأنواع (Type Errors)

- **TypeError**: تحدث بسبب محاولة تحويل أو معاملة متغير كأنه من نوع آخر غير متوافق معه في الذاكرة (مثلاً: إجبار متغير Object يحمل نصاً على أن يُعامل كـ int بدون تحويل صريح).

٥. أخطاء القيم الفارغة والتهيؤ (Null & Initialization)

- **LateInitializationError**: تحدث بسبب محاولة استخدام وقراءة متغير تم تعريفه بـ late قبل أن يتم إسناد أي قيمة له.
- **NullThrownError / TypeError (Null Safety Related)**: تحدث بسبب استخدام معامل التوكيد ! على متغير قيمته الفعلية هي null في تلك اللحظة.

٦. أخطاء الاتصالات والإدخال/الخروج (I/O & Network)

- **SocketException**: يحدث في تطبيقات الشبكة عند فشل الاتصال بالإنترنت أو عدم التمكن من الوصول للسيرفر.
- **FileSystemException**: الـ File System: يحدث عند محاولة قراءة أو كتابة ملف غير موجود، أو لا تملك الصلاحيات الكافية للوصول إليه.

٧. أخطاء الذاكرة والنظام (System Errors)

- **StackOverflowError**: يحدث عادة عند استدعاء الدالة لنفسها (Recursion) إلى المالا نهاية، مما يؤدي إلى امتلاء ذاكرة الاستدعاء (Stack).

رمي الاستثناء (throw): هو إجراء يتعمده المطور لتوليد خطأ أثناء التشغيل (Explicit Exception) عند إخلال البرنامج بشرط معين أو منطق عمل محدد (Business Logic)، وذلك لإيقاف التنفيذ العادي فوراً ونقل التحكم إلى كتلة الالتقاط (catch)

❖ **الرمي المباشر (throw):** الكلمة throw تقوم بإنشاء ورمي كائن استثناء من نوع `FormatException`، النتيجة المباشرة: يقطع البرنامج الدورة فوراً، ويتوقف تنفيذ باقي حلقة `for` ولن يُطبع `i=5` وما بعدها.

❖ **الاستقبال والتنفيذ (catch):** يخرج البرنامج من كتلة `try` فوراً ويبحث عن كتلة `catch`، تلتقط كتلة `catch (e)` الاستثناء المرفوع وتقوم بطباعته عبر `print(e)`.

الغرض والفائدة من استخدامه:

١. فرض الشروط والتحقق (Validation):
منع استكمال تنفيذ الكود إذا كانت البيانات أو حالة النظام غير صالحة (مثلاً: إيقاف المعاملة إذا كان رصيد السحب أكبر من المتاح).
٢. التنبيه المبكر (Fail Fast): اكتشاف المخالفات المنطقية فور حدوثها وتمريضها لمعالج الأخطاء بدلاً من ترك الكود يعمل بنتائج خاطئة.

```
void main() {
  try {
    for (int i = 0; i <= 10; i++){
      if (i == 5) {
        // رمي استثناء
        throw FormatException;
      }
      print("i=$i");
    }
  }
  // استقبال وتنفيذ الاستثناء
  catch (e) {print(e);
}
```

```
void main() {
  showinfo();
}
// يمكن رمي الاستثناء من خلال الدوال
showinfo() {
  try {
    for (int i = 0; i <= 10; i++) {
      if (i == 5) {
        throw FormatException;
      }
      print("i=$i");
    }
  }
  catch (e) { print(e); }
```


Thank you